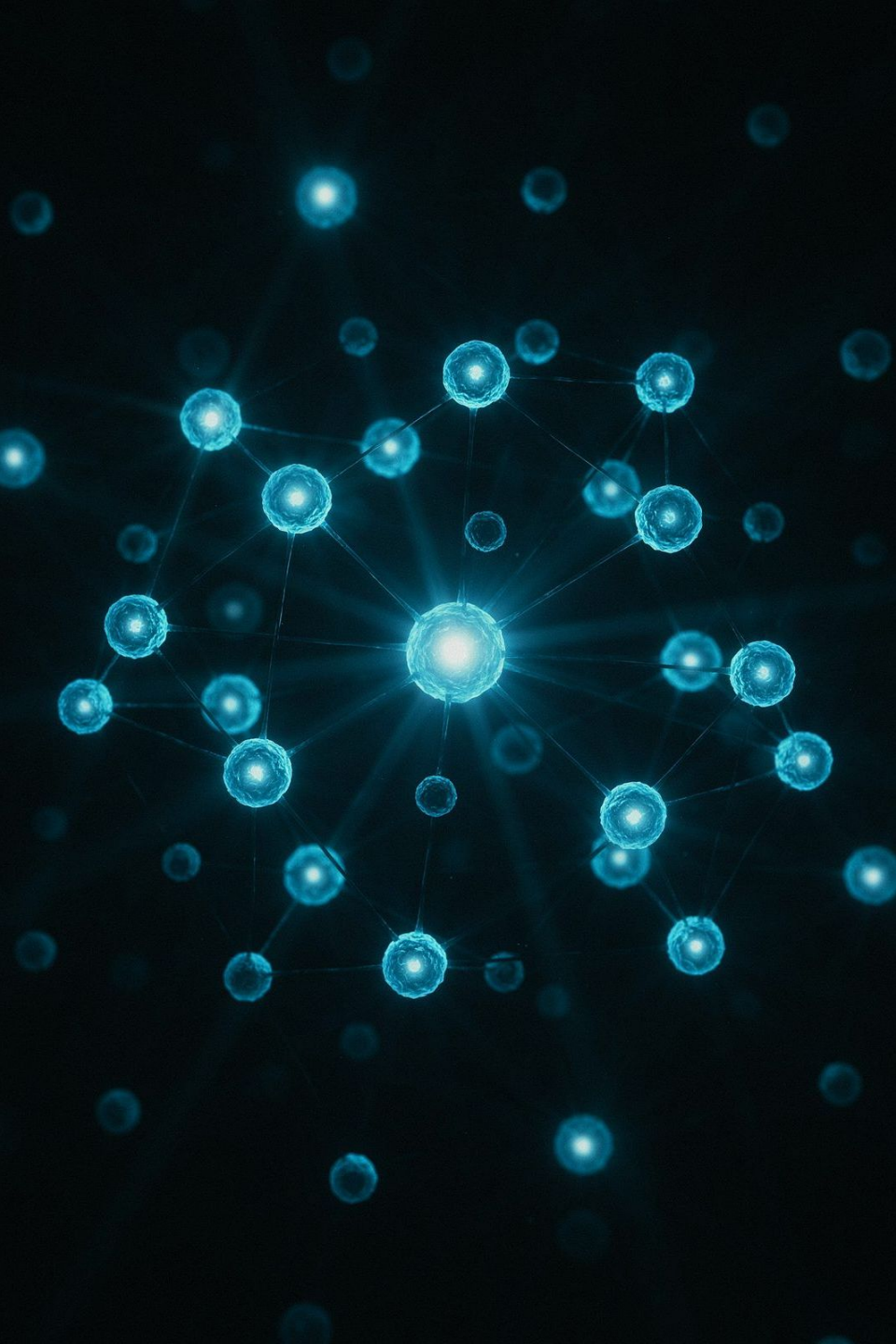




S10.3 · Entrenamiento y Regularización

Guía práctica para entrenar redes neuronales de forma robusta: diagnóstico de sobreajuste, técnicas de regularización, control de gradientes y herramientas de monitorización. Todo lo que necesitas para llevar tus modelos de la experimentación a la producción.

MACHINE LEARNING

REDES NEURONALES

NIVEL INTERMEDIO

Contenidos de la sesión

Un recorrido completo por las técnicas esenciales de entrenamiento y regularización, organizadas en nueve bloques temáticos.

01

Sobreajuste en redes

Causas y síntomas del overfitting

02

Curvas de aprendizaje

Diagnóstico visual del entrenamiento

03

Dropout y L2

Regularización estándar

04

Early stopping

Parada anticipada óptima

05

Batch normalization

Estabilización del entrenamiento

06

LR e inicialización

Hiperparámetros críticos

07

Gradientes problemáticos

Vanishing y exploding gradient

08

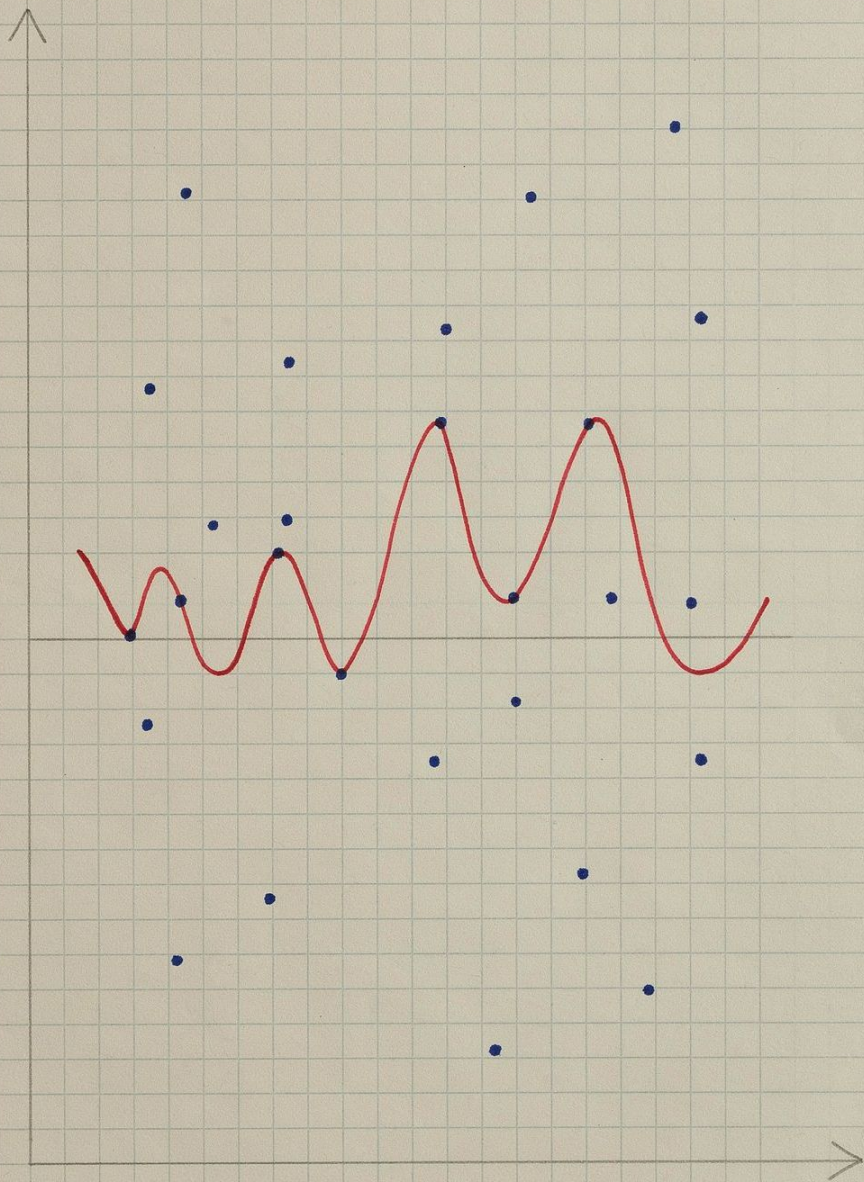
Callbacks y TensorBoard

Monitorización en tiempo real

09

Checklist de diagnóstico

Protocolo de revisión completo



BLOQUE 1

Sobreajuste en Redes Neuronales

El sobreajuste ocurre cuando el modelo aprende el **ruido** de los datos de entrenamiento en lugar de los patrones subyacentes. La red memoriza en vez de generalizar, obteniendo un rendimiento excelente en entrenamiento pero pobre en datos nuevos.

¿Por qué sobreajenan las redes?

Las redes neuronales profundas tienen millones de parámetros y una capacidad expresiva enorme. Sin controles adecuados, pueden ajustarse perfectamente a cualquier conjunto de datos, incluso uno aleatorio.



Capacidad excesiva

Demasiados parámetros respecto al volumen de datos disponibles.



Datos insuficientes

Conjunto de entrenamiento pequeño o poco representativo.



Entrenamiento prolongado

Demasiadas épocas sin mecanismos de parada.



Sin regularización

Ausencia de técnicas que penalicen la complejidad del modelo.



La brecha entre la pérdida de entrenamiento y la de validación es el indicador principal de sobreajuste.

Leer las Curvas de Aprendizaje

Las curvas de aprendizaje representan la evolución de la pérdida (y otras métricas) a lo largo de las épocas, tanto en el conjunto de entrenamiento como en el de validación. Son la herramienta de diagnóstico más inmediata y potente disponible durante el entrenamiento.

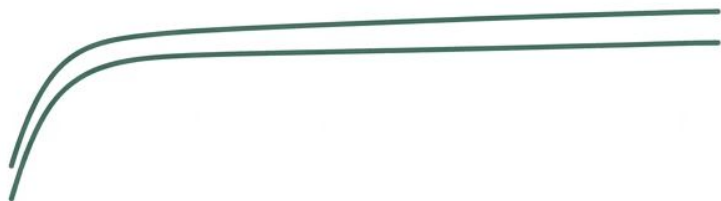
Ajuste ideal



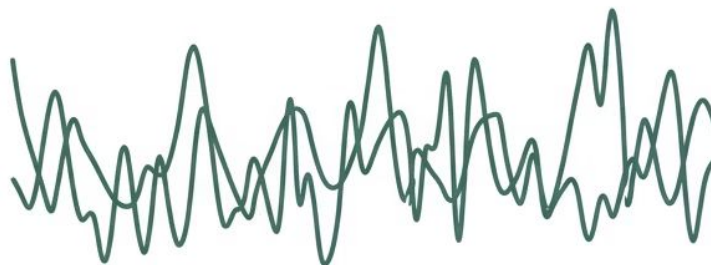
Sobreajuste



Subajuste



Alta varianza



Interpretación práctica de las curvas

Señales de alarma

- La pérdida de validación sube mientras la de entrenamiento baja → **overfitting**
- Ambas pérdidas estancadas desde épocas tempranas → **underfitting**
- Oscilaciones muy pronunciadas → tasa de aprendizaje demasiado alta
- Descenso muy lento y monótono → tasa de aprendizaje demasiado baja

Buenas señales

- Ambas curvas descienden de forma paralela y suave
- La brecha entre train y val se mantiene pequeña y estable
- La pérdida de validación sigue bajando al final del entrenamiento
- El modelo converge sin oscilaciones bruscas

BLOQUE 3

Dropout

El **Dropout** desactiva aleatoriamente un porcentaje de neuronas en cada paso de entrenamiento. Esto obliga a la red a aprender representaciones redundantes y distribuidas, reduciendo la codependencia entre neuronas y actuando como un ensamblado implícito de modelos.



Dropout: configuración y uso correcto

Cómo aplicarlo en Keras

```
from tensorflow.keras.layers import Dropout

model.add(Dense(256, activation='relu'))
model.add(Dropout(0.4)) # 40% de neuronas desactivadas
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(10, activation='softmax'))
```

Durante la inferencia, Dropout se desactiva automáticamente y los pesos se escalan para compensar.

Reglas prácticas

- Capas densas profundas: **0.3 – 0.5**
- Capas convolucionales: **0.1 – 0.25**
- No aplicar en la capa de salida
- Aumentar la tasa si hay overfitting severo
- Reducirla si el entrenamiento es muy lento

Regularización L2 (Weight Decay)

La regularización L2 añade un término de penalización a la función de pérdida proporcional al cuadrado de los pesos. Esto desincentiva valores de pesos muy grandes, produciendo modelos más suaves y generalizables. También se conoce como *weight decay*.

Función de pérdida con L2

$$\mathcal{L}_{total} = \mathcal{L}_{original} + \lambda \sum_i w_i^2$$

El hiperparámetro λ controla la intensidad de la penalización.
Valores típicos: 1e-4 a 1e-2.

Implementación en Keras

```
from tensorflow.keras.regularizers import l2

model.add(Dense(256,
    activation='relu',
    kernel_regularizer=l2(1e-3)))
```

L2 puede combinarse con Dropout para una regularización más fuerte.



BLOQUE 4

Early Stopping

El **early stopping** detiene el entrenamiento automáticamente cuando la métrica de validación deja de mejorar durante un número determinado de épocas (*patience*). Es una de las técnicas de regularización más eficaces y con menor coste computacional, al evitar también iteraciones innecesarias.

Implementación de Early Stopping

```
from tensorflow.keras.callbacks import (  
    EarlyStopping, ModelCheckpoint  
)  
  
es = EarlyStopping(  
    monitor='val_loss',  
    patience=10,  
    restore_best_weights=True,  
    min_delta=1e-4  
)  
  
mc = ModelCheckpoint(  
    'best_model.h5',  
    monitor='val_loss',  
    save_best_only=True  
)  
  
model.fit(X_train, y_train,  
    epochs=200,  
    callbacks=[es, mc])
```

Parámetros clave

monitor

Métrica observada: `val_loss` o `val_accuracy`

patience

Épocas sin mejora antes de parar. Típico: 5–20

restore_best_weights

Recupera los pesos del mejor época automáticamente

min_delta

Mejora mínima para considerar progreso real

BLOQUE 5

Batch Normalization

La **Batch Normalization** normaliza las activaciones de cada capa dentro del mini-batch durante el entrenamiento, manteniéndolas con media cero y varianza unitaria. Esto estabiliza el proceso de aprendizaje, permite usar tasas de aprendizaje más altas y actúa como regularizador implícito.



Batch Normalization: mecanismo y uso

¿Qué hace internamente?

Para cada mini-batch, BatchNorm calcula la media y varianza de cada característica y las normaliza. Después aplica dos parámetros entrenables, γ (escala) y β (desplazamiento), para recuperar la expresividad del modelo.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

Uso en Keras y buenas prácticas

```
from tensorflow.keras.layers import (  
    Dense, BatchNormalization, Activation  
)  
  
# Orden recomendado: Linear → BN → Activación  
model.add(Dense(256, use_bias=False))  
model.add(BatchNormalization())  
model.add(Activation('relu'))
```

- Colocar **antes** de la activación (debate abierto)
- Suele permitir eliminar Dropout en algunas capas
- Usar `use_bias=False` en la capa densa previa

Tasa de Aprendizaje e Inicialización

La **tasa de aprendizaje** (learning rate) es el hiperparámetro más influyente en el entrenamiento. Una LR muy alta provoca divergencia; muy baja, convergencia excesivamente lenta. La correcta **inicialización de pesos** determina el punto de partida y puede facilitar o dificultar radicalmente el aprendizaje.



Inicialización He (ReLU)

Escala los pesos según $\sqrt{2/n_{in}}$. Ideal para activaciones ReLU y variantes. Es el estándar en redes profundas modernas.



Inicialización Glorot / Xavier

Escala según $\sqrt{2/(n_{in} + n_{out})}$. Adecuada para activaciones simétricas como `tanh` o `sigmoid`.



Learning Rate Schedulers

Reducir la LR durante el entrenamiento (step decay, cosine annealing, ReduceLROnPlateau) permite convergencia más fina y mejores resultados finales.

Estrategias de Learning Rate

```
from tensorflow.keras.callbacks import (
    ReduceLROnPlateau
)
from tensorflow.keras.optimizers.schedules import (
    CosineDecay
)

# Opción 1: Reducir LR al estancarse
rlrop = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=5,
    min_lr=1e-6
)

# Opción 2: Cosine Annealing
schedule = CosineDecay(
    initial_learning_rate=1e-3,
    decay_steps=1000
)
optimizer = Adam(learning_rate=schedule)
```

Regla de oro

Empieza con $lr=1e-3$ con Adam. Si el entrenamiento es inestable, reduce a $1e-4$. Usa schedulers para afinar la convergencia final.

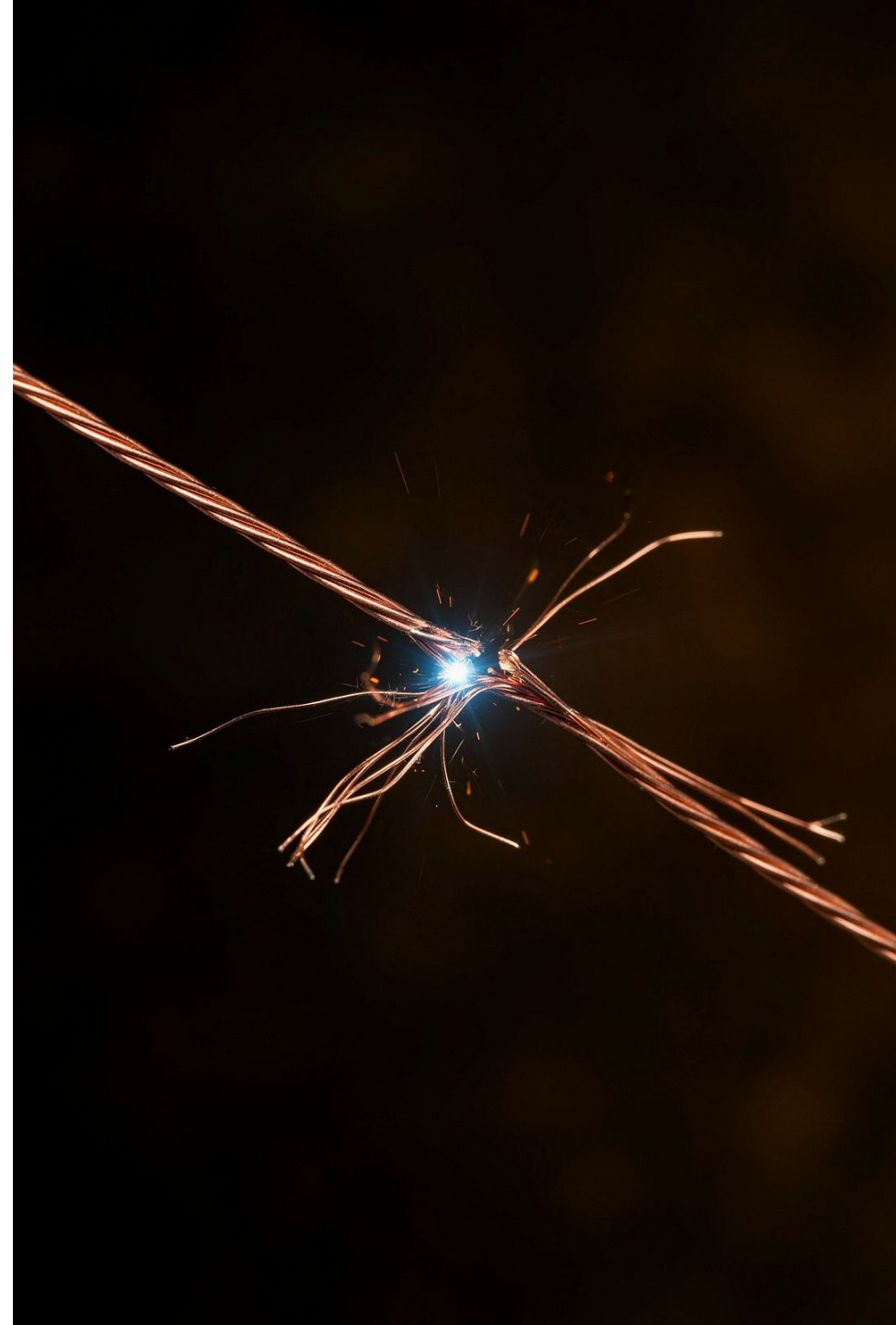
LR Finder

Técnica que incrementa la LR exponencialmente durante unas pocas épocas para identificar el rango óptimo antes de entrenar en serio. Disponible en la librería `keras-lr-finder`.

BLOQUE 7

Vanishing y Exploding Gradient

Durante la retropropagación, los gradientes se multiplican capa a capa. En redes profundas, esto puede causar que los gradientes se anulen (*vanishing*) o crezcan exponencialmente (*exploding*), impidiendo el aprendizaje efectivo en las capas más alejadas de la salida.



Diagnóstico y soluciones

Vanishing Gradient

Síntoma: Las capas iniciales casi no aprenden; los pesos apenas cambian.

- Usar activaciones **ReLU** o **Leaky ReLU** en lugar de sigmoid/tanh
- Inicialización He para pesos
- Batch Normalization entre capas
- Conexiones residuales (ResNet)

Exploding Gradient

Síntoma: La pérdida diverge o aparece NaN durante el entrenamiento.

- **Gradient Clipping:** limitar la norma del gradiente
- Reducir la tasa de aprendizaje
- Inicialización correcta de pesos
- Batch Normalization

```
# Gradient clipping en Keras/TF
```

```
optimizer = Adam(learning_rate=1e-3, clipnorm=1.0)
```

```
# o por valor: clipvalue=0.5
```



BLOQUE 8

Callbacks y TensorBoard

Los **callbacks** son funciones que se ejecutan automáticamente en momentos concretos del entrenamiento (al final de cada época, al mejorar una métrica, etc.). **TensorBoard** es la herramienta de visualización estándar para monitorizar experimentos de deep learning en tiempo real.

Callbacks esenciales y TensorBoard

```
import tensorflow as tf
from tensorflow.keras.callbacks import (
    EarlyStopping, ModelCheckpoint,
    ReduceLROnPlateau, TensorBoard
)
import datetime

log_dir = "logs/" + datetime.datetime.now().strftime(
    "%Y%m%d-%H%M%S"
)

callbacks = [
    EarlyStopping(monitor='val_loss', patience=10,
                  restore_best_weights=True),
    ModelCheckpoint('best.h5', save_best_only=True),
    ReduceLROnPlateau(factor=0.5, patience=5),
    TensorBoard(log_dir=log_dir, histogram_freq=1)
]

model.fit(X_train, y_train,
          validation_data=(X_val, y_val),
          epochs=100,
          callbacks=callbacks)

# Lanzar TensorBoard:
# tensorboard --logdir logs/
```

¿Qué monitorizar en TensorBoard?



Scalars

Pérdida y métricas por época en
train y validación



Graphs

Arquitectura del modelo como
grafo computacional



Histograms

Distribución de pesos y gradientes
por capa



Learning rate

Evolución de la tasa de aprendizaje
con schedulers

Checklist de Diagnóstico

Antes de declarar un modelo «listo», recorre esta lista sistemáticamente. Un diagnóstico estructurado ahorra horas de búsqueda de errores.

1) Curvas de aprendizaje - overfitting y convergencia

2) Brecha train/val - menor de 5%

3) Regularización aplicada - Dropout, L2

4) Gradientes - verificar NaN/extremos

5) Tasa de aprendizaje - scheduler, LR final razonable

6) Inicialización - He/ReLU, Glorot/tanh

7) Batch Normalization - presente en redes profundas

8) Early stopping + checkpoint - modelo guardado

9) TensorBoard - experimento registrado

Resumen: las siete reglas de oro

Si tuvieras que recordar solo siete principios de esta sesión para cualquier proyecto de deep learning, estos serían los más impactantes.

- 1 Lee siempre las curvas de aprendizaje**
Son la radiografía del entrenamiento. No tomes decisiones sin consultarlas.
- 2 Regulariza desde el principio**
Añade Dropout y/o L2 antes de que el overfitting aparezca, no después.
- 3 Usa Early Stopping + ModelCheckpoint siempre**
Son gratuitos computacionalmente y protegen contra el sobreentrenamiento.
- 4 Batch Normalization en redes profundas**
Estabiliza el entrenamiento y suele mejorar los resultados sin coste adicional.
- 5 Inicializa correctamente y controla la LR**
He + Adam con `lr=1e-3` es un punto de partida sólido para casi todo.
- 6 Vigila los gradientes**
Activa `histogram_freq=1` en TensorBoard para detectar problemas temprano.
- 7 Registra todos tus experimentos**
TensorBoard + nombres de run con timestamp evitan la confusión experimental.